Design Document

AI-Powered Supply Chain Disruption Assistant — semantic retrieval & AI-assisted disruption analysis

1. Problem Statement

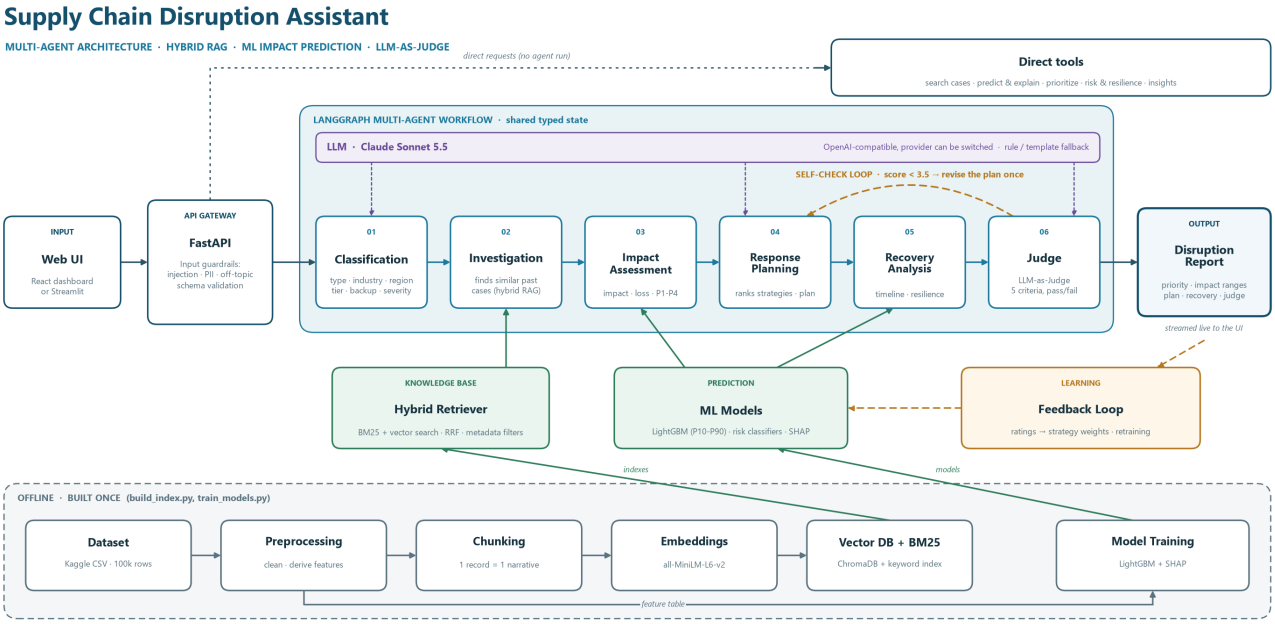
Supply chain teams regularly face disruptions caused by supplier issues, natural events, operational failures, labor problems and geopolitical events, leading to production losses, financial impact, delayed recovery and supplier changes. To decide how to respond, operations teams must find similar past disruptions, understand their impact, evaluate which responses worked and estimate recovery - today by manually searching historical records, which is slow and inconsistent.

Goal: an AI assistant that answers, in seconds and with evidence: Has a similar disruption occurred, and how was it resolved? Which factors drive production impact, financial loss and recovery? Which response strategy is most effective for this disruption, and how fast will operations recover? How urgent is it relative to other open disruptions, and how resilient is the affected supplier?

Requirements coverage

Requirement	Where implemented
Basic RAG similarity search	src/rag/vector_store.py, src/rag/hybrid_retriever.py — /search
Hybrid keyword + semantic search	src/rag/keyword_index.py (BM25) + weighted RRF fusion in hybrid_retriever.py
Classification agent	src/agents/classifier_agent.py — /classify
Metadata filtering (type, industry, tier, region, severity)	filter mask applied to both retrievers before ranking (Filters schema)
Response & recovery recommendations	src/services/recommendation.py, agents/response_agent.py, agents/recovery_agent.py — /recommend
Input validation guardrails	src/api/schemas.py (Pydantic), src/services/guardrails.py
Prioritization (severity, impact, loss)	src/services/prioritization.py — /prioritize
API endpoints	src/api/routes.py, src/main.py (FastAPI) — 20 endpoints
DeepEval	evaluation/deepeval_suite.py (offline), src/services/deep_check.py — /analyze/deep-check (per analysis)
Impact / loss / recovery prediction models	src/services/ml_models.py — /predict
LLM-as-Judge	src/agents/judge_agent.py — /judge, Judge node in workflow
Multi-agent workflow	src/agents/workflow.py (LangGraph) — /analyze, /analyze/stream
Resilience & disruption-risk prediction	src/services/risk_resilience.py — /risk
Explainable factor analysis	src/services/explainability.py — /explain, /insights
Feedback loop	src/services/feedback.py — /feedback, /feedback/stats, /feedback/retrain
Front-end	frontend.py (Streamlit, 7 tabs) and web/ (React + TypeScript, 7 pages)

2. Architecture Design



The system is a single Python microservice (FastAPI) with three layers:

- **Offline pipeline** - the CSV is cleaned and enriched, each record is turned into a narrative chunk, embedded into ChromaDB and indexed with BM25; LightGBM models and SHAP explainers are trained on the full dataset. Everything is written to artifacts/ by two one-time scripts (build_index, train_models).
- **Online layer** - every request passes input guardrails; the hybrid retriever fetches evidence; the ML models predict impact and recovery; the LLM (Claude Sonnet 5.5 through Anthropic's OpenAI-compatible endpoint; any OpenAI-compatible provider can be selected) is used for classification, plan wording, judging and narratives, each with a deterministic fallback; the LangGraph workflow orchestrates the agents over a shared state.
- **Serving layer** - 20 REST endpoints consumed by two front ends (Streamlit and React; the React UI uses the streaming endpoint to show each agent finishing live). Feedback is stored in SQLite and fed back into ranking, prompting and retraining; offline evaluation results are exposed via /evaluation.

At startup the API loads the indexes, the embedding model and the ML models and runs one retrieval and one prediction (no LLM), so the first user request does not pay the initialisation cost (about 4.5 s for a full analysis in template mode, measured right after startup).

Data flow of a disruption analysis (/analyze)

Step	Component	Input -> Output
1	Guardrails	raw text -> sanitized text (PII masked) or 422 rejection
2	Classification Agent	text -> type, industry, region, tier, size, backup, severity (+ where the severity came from), confidence
3	Investigation Agent	text + attributes -> cleaned retrieval query, filters with a severity band of ±1, hybrid search with step-by-step relaxation -> 10 closest cases, a pool of comparable cases for strategy comparison, completed profile with stated assumptions
4	Impact Assessment Agent	profile -> predicted impact/loss/recovery (P10-P90), SHAP drivers, priority P1-P4
5	Response Planning Agent	profile + comparable pool -> what-if per strategy, ranked strategies (Alternative Supplier ranked last when there is no backup), plan written by the LLM from a fact sheet, checked by output guardrails
6	Recovery Analysis Agent	profile + strategy -> timeline & milestones, supplier-change risk, resilience, KPIs
7	Judge Agent	evidence + answer -> 5 criterion scores, pass/fail; on fail -> one revision of step 5
8	Report	executive summary + full structured JSON for the UI

3. Technology Selection

Component	Choice	Justification
LLM	Claude Sonnet 5.5 (claude-sonnet-5-5) via Anthropic's OpenAI-compatible endpoint; LLM_PROVIDER switch	Reliable structured (JSON) output and close adherence to the evidence it is given, which the classification, plan and judge prompts depend on. Anthropic's endpoint ignores OpenAI JSON mode, so the client falls back to prompt-only JSON and validates every field. Used only where language understanding or generation adds value; all numbers come from retrieval and ML models, never from the LLM. Provider-agnostic: NVIDIA, Gemini, AICredits (gpt-5-nano) and Ollama also work, and the provider can be switched at runtime.
Embedding model	sentence-transformers/all-MiniLM-L6-v2	Free, local (no data leaves the machine, no per-call cost), 384-dim vectors keep a 100k-vector index small (about 150 MB in memory); strong quality/speed trade-off for short English descriptions.
Vector database	ChromaDB (persistent) + exact in-memory search for filtered queries	Embedded (no server to run), persists to disk, cosine HNSW for fast unfiltered search. Chroma's metadata filtering over 100k rows took 0.8-6 s per query, so filtered queries rank the allowed rows with a NumPy matrix product over the same vectors instead (~0.1 s, exact).
Keyword search	BM25 on scikit-learn sparse matrices	Exact term matching for labels and domain words; precomputed sparse weights rank 100k documents in milliseconds.
Agent framework	LangGraph	Explicit state graph with conditional edges (guardrail stop, judge-driven revision loop), typed shared state, streaming of node updates, deterministic and debuggable control flow.
ML	LightGBM + SHAP	Strong on tabular data, native categorical handling, quantile objective for prediction intervals; TreeSHAP gives exact, fast explanations. Benchmarked against random forest and linear models (equal accuracy).
API / UI	FastAPI + Streamlit, React (Vite + TypeScript)	Typed validation via Pydantic (doubles as guardrails) and auto OpenAPI docs; Streamlit is the required Python UI; the React dashboard adds live agent progress, report download and history.
Evaluation	DeepEval + RAGAS + custom metrics	Two independent LLM-evaluation frameworks plus label-based retrieval, classification and model metrics that need no LLM.

4. Retrieval Strategy

Chunking strategy

The dataset is fully structured (no free text), so document splitting does not apply. Each record becomes exactly one chunk: a ~90-word narrative that verbalises every field (severity word + level, type with synonym descriptors, tier, size, industry, region, backup availability, impact, loss bucket, response, timing, recovery, supplier change). One record per chunk keeps each retrieved unit a complete, citable case (its ID is the chunk ID), and the synonym descriptors bridge vocabulary between how users describe incidents and the dataset's labels.

Embedding approach

Narratives are embedded with all-MiniLM-L6-v2 (normalised, 384-dim) and stored in ChromaDB with the structured fields as metadata; the same vectors are cached as a NumPy matrix for filtered search. Queries are embedded with the same model; similarity is cosine.

Retrieval method and hybrid search

- Semantic: nearest neighbours by cosine similarity (pool of 200 candidates); ChromaDB HNSW without filters, exact matrix search over the allowed rows with filters.
- Keyword: BM25 (k1=1.5, b=0.75, unigrams+bigrams, stop words removed) over the same narratives, with the same filters applied as a mask.
- Fusion: weighted Reciprocal Rank Fusion, score = α/(60+rank_sem) + (1-α)/(60+rank_kw), normalised to 0-1; α (default 0.6) is adjustable per request. RRF is robust because BM25 and cosine scores are on different scales. Modes: hybrid, semantic only, keyword only.
- Metadata filters: disruption type, industry, supplier tier, region, size, response, backup, exact severity or severity range, applied before ranking so results are the best matches inside the filter.
- Agent retrieval: the investigation agent searches with the first sentence of the description (the event) and removes strategy words from later sentences, so constraints like 'we have 10 days of inventory' do not pull in cases that merely used that strategy. It filters by the classified type, industry and region and a severity band of ±1, and when fewer than 5 cases match it relaxes step by step (region, then a ±2 band, then industry, then severity), recording each relaxation. Filters the user set are never relaxed.
- Feedback boosts: cases cited in highly rated answers receive a bounded score multiplier (0.8-1.2).

Measured effect: on the 30 golden scenarios Precision@10 is 0.36 for keyword, 0.50 semantic, 0.54 hybrid and 0.92 hybrid with automatic filters; on 180 everyday queries hybrid reaches 0.83 with a median latency of 60 ms.

5. Agent Design

Agent	Role	Tools / logic	Output
Guardrails	Block unsafe or off-topic input	regex injection patterns, PII masking, profanity, domain rule (type keyword, or 2 supply-chain terms, or 1 term + an incident word; LLM topic check as tie-breaker)	allowed flag, sanitized text
Classification	Understand the incident	LLM JSON extraction validated against allowed values; keyword rule engine fallback that reads stated durations for severity	structured attributes, severity source, confidence
Investigation	Find precedent	cleaned retrieval query, hybrid retriever, severity band, filter relaxation, comparable-case pool, profile completion	similar cases, comparable pool, assumptions
Impact Assessment	Quantify impact & urgency	LightGBM median/quantile models, SHAP, priority scorer	predictions with ranges, drivers, P1-P4
Response Planning	Choose and explain the response	strategy stats on the comparable pool, ML what-if per strategy, feedback multipliers, LLM plan from a fact sheet, output guardrails	ranked strategies, action plan, close-call / minor-event notes
Recovery Analysis	Plan the recovery	recovery models for the chosen strategy, supplier-change classifier, resilience assessment	timeline, milestones, KPIs, resilience
Judge	Validate the answer	LLM-as-Judge on 5 criteria with a strict rubric (checklist fallback)	scores, reasons, pass/fail, feedback

Workflow and agent communication

Agents are LangGraph nodes sharing a typed WorkflowState (query, overrides, filters, classification, investigation, impact, response, recovery, judgement, revisions, trace). Each node reads what it needs from the state and writes its own key, so agents are decoupled and individually testable. Conditional edges implement control: the guardrail node can end the run; after the judge, a failing score (overall < 3.5 or faithfulness < 3) routes back to Response Planning with the judge's feedback added to the prompt (at most one revision), otherwise to the report. Every node appends a trace entry (agent, status, duration, summary); /analyze/stream sends each entry to the UI as it happens.

How the response strategy is chosen

Ten closest cases are too few to count which strategy works best, and the search wording can skew them. So strategies are compared on a pool of comparable past cases chosen by profile only: same type, industry and region, severity within ±1 and the same backup-supplier status (up to 1,000 cases; at least 30, else the closest cases are used). Strategies are used very differently with and without a backup supplier, so mixing the two would credit a strategy for the situations it happens to be used in. For each strategy the expected recovery, loss and supplier-change risk average the pool's history and the ML what-if prediction for this profile, and:

```
score = 100 x (0.45 x faster full recovery + 0.25 x lower revenue loss + 0.15 x lower supplier-change risk
            + 0.15 x evidence (cases that used it, capped at 5))  x  feedback multiplier (0.85-1.15)
```

Feasibility rule: when the profile has no backup supplier, Alternative Supplier needs a qualified second source the user does not have, so it is not an immediate option. Its score is halved and it is ranked after every strategy that can start now, with a note saying why; it can be neither the recommendation nor the fallback. The data records Alternative Supplier in no-backup cases too (a new source was found later), and on recovery speed alone it often scored highest, which produced 'switch supplier' advice for users who had just said they had no backup. The fact sheet lists it as not an immediate option, the plan prompt forbids telling the user to activate a backup they do not have (qualifying a new source may appear only as a parallel, long-lead step), and the judge deducts points if a plan does.

The top-scoring strategy is the recommendation; the LLM never chooses it. When the top two are within 5 points the plan carries a 'close call' note, and a severity 1-2 event with priority P3/P4 gets a 'minor event' note.

Grounding the LLM's plan

- The LLM receives a fact sheet: the profile, the priority level and response window, past outcomes per strategy in the comparable pool, model estimates for the chosen and fallback strategies, the headline impact and loss estimates, and the closest example cases. For a minor event (severity 1-2, priority P3/P4) it also gets a scale note: the user's own description of duration or loss takes precedence and the estimates are only an upper bound.
- Plan rules: answer the user's question first, build on what the user stated (stock cover, partial backup capacity, a restart under way), give every action an owner and a deadline plus at least one numeric trigger for escalating, and include the quality or regulatory step in pharmaceuticals and aerospace.
- Output guardrails: case IDs not in the evidence are removed; a rationale stating numbers not in the fact sheet, an unfinished sentence, or a 'historical' claim about a strategy no comparable case used is replaced by the template; leaked internal field names are turned into plain words; blank list items are dropped. Every intervention is recorded in a guardrail note.
- Assumptions are explicit: any field the user did not state is filled from similar cases and listed, and the judge deducts if the plan presents an assumption as fact.

Prioritization model

score = 100 x (0.35·severity + 0.25·production impact + 0.25·log financial loss + 0.10·tier factor + 0.05·no backup), all factors normalised to 0-1 (loss against the 1st-99th percentile of historical log-loss; tier factor 1 / 0.75 / 0.5 / 0.25 for tiers 1-4). Levels: P1 Critical >= 70, P2 High >= 50, P3 Medium >= 30, else P4 Low. For new incidents with unknown impact or loss, the ML predictions are used. The factor breakdown is returned so every priority is explainable.

Explainability

Three complementary views: (1) global SHAP importance per model, computed at training time; (2) local SHAP contributions for each prediction, reported in days, percentage points or % change in loss (and in probability points for yes/no outcomes in the UI); (3) statistical factor analysis on the full dataset - group means with one-way ANOVA / Welch t-tests and eta-squared effect sizes, plus severity-adjusted effects, turned into plain-language insights with a caution where a raw difference is mostly explained by severity.

Feedback loop

Ratings, comments, the strategy actually chosen and real outcomes are stored in SQLite. They shift strategy multipliers (±5% per rating point above or below neutral, damped for few votes, bounded 0.85-1.15), boost or demote cited cases (bounded 0.8-1.2), provide few-shot style examples from 5-star plans, and reported outcomes can be added to the training data via /feedback/retrain.

6. Evaluation Strategy

Layer	Method	Metrics
Retrieval	30 golden scenarios + 180 everyday queries, label-based relevance	Precision@10, Hit@1, Hit@10, MRR, nDCG@10, latency, filter compliance
Retrieval bias	paired test with and without the word 'supplier'	share of Alternative Supplier cases in the top 10
Classification	golden attributes	accuracy per attribute, severity MAE, within ±1
Generation (DeepEval)	custom OpenAI-compatible model adapter	Answer Relevancy, Faithfulness, Contextual Precision/Recall, G-Eval Recommendation Quality & Analysis Correctness
Generation (RAGAS)	ragas.metrics.collections	Faithfulness (whole plan and explanation), Answer Relevancy, Context Precision, Context Recall
LLM-as-Judge	in-workflow validator	5 criteria (1-5), pass rate
Strategy choice	acceptable-strategy set from the golden reference	share of scenarios with an acceptable strategy
ML models	20% hold-out + benchmark	MAE, RMSE, R^2, baseline MAE, interval coverage, ROC-AUC, PR-AUC, F1
Software	pytest (offline), 74 tests	guardrails, retrieval, models, API contracts, workflow, investigation, feedback
Per analysis	optional deep check in the UI	DeepEval relevancy, faithfulness and two G-Eval rubrics with reasons

Results summary (details in docs/Evaluation_Report.pdf)

- LLM-based results are from the Claude Sonnet 5.5 run (10 scenarios, Claude as generator and evaluator).
- Classification (Claude): categorical attributes 100%, severity MAE 0.40
- DeepEval: answer relevancy 1.00, faithfulness 1.00, contextual precision 0.87, contextual recall 0.47, recommendation quality 0.56, analysis correctness 0.84
- RAGAS: faithfulness 0.42, faithfulness explanation 0.86, answer relevancy 0.70, context precision 0.65, context recall 0.47
- LLM-as-Judge: mean 3.48/5, pass rate 60% on the golden set
- LLM-as-Judge on the 10 demo scenarios after the plan-writer improvements: mean 3.92/5, 10 of 10 pass
- Models: production impact R^2 0.70, full recovery R^2 0.40, high-impact AUC 0.92

7. Challenges and Trade-offs

- **Structured data in a RAG system.** With no free text, retrieval quality depends on how records are verbalised. Narratives with synonym descriptors plus metadata filters work well; similarity alone conflates cases with similar wording but different industries or severities, which is why the agent filters and uses a severity band.
- **Keeping the LLM away from decisions and numbers.** A language model asked to pick a strategy and quote figures can invent numbers or contradict the headline estimates. The strategy is therefore chosen by scoring, the LLM only explains it from a fact sheet that carries the same numbers as the headline, and output guardrails replace any unsupported number or citation.
- **Confounded strategy outcomes.** In the data, the chosen strategy depends on the situation (for example Alternative Supplier is used far more when a backup exists; Combined Strategy mostly at severity 4-5). Ranking strategies on the 10 closest cases also let the query wording decide. The comparable-case pool (same profile, same backup status) fixes both. Feasibility is a separate question the outcome data cannot answer: Alternative Supplier still looked fastest in no-backup pools, so a rule now keeps it out of the

recommendation when there is no qualified backup. Outcomes remain observational, so recommendations are decision support, not causal guarantees.

- **Severity from free text.** Severity drives filters, predictions and priority but is rarely stated. The classifier reads stated durations ('a 2-day outage'), ignores stock runway and forecasts, records where the value came from, and lists a defaulted severity as an assumption.
- **Retrieval wording bias.** Narratives end with the response label, so 'supplier' in a query pulls Alternative Supplier cases (measured with a paired test). It does not affect the recommendation (pool chosen by profile); indexing only the situation part of each narrative is the planned fix.
- **Latency.** ChromaDB metadata filtering cost 0.8-6 s per query; exact search over cached vectors brought it to ~0.1 s. A startup warm-up removed a 5-15 s first-request delay. A full analysis with Claude takes about 20-25 s, almost all of it LLM time; template mode takes about 4.5 s.
- **Index build time.** Embedding 100k narratives on a CPU takes 15-60 minutes (one-time). A stratified INDEX_SAMPLE_SIZE option trades recall on rare filter combinations for a much faster build.
- **Predictability limits.** Production impact is well explained by the inputs (R^2 ~0.7); revenue loss and recovery time are noisier and supplier change is weakly predictable. LightGBM, random forest and linear models score the same, so the data is the ceiling. The system shows P10-P90 ranges and probabilities, not single numbers.
- **Synthetic-looking data.** At equal severity the disruption type has almost no effect on outcomes; raw differences between types come mostly from their different severity mix. The Insights and Risk pages show severity-adjusted views and caution notes so users are not misled.
- **Evaluating with an LLM.** Claude generated and evaluated the answers, and only 10 scenarios were LLM-scored for cost. Label-based retrieval, classification, strategy and model metrics provide an LLM-independent check.
- **Dependency friction.** ragas 0.4.3 imports a module removed from langchain-community 0.4; a scoped placeholder avoids downgrading LangChain (needed by LangGraph 1.x). DeepEval must be imported on the main thread at startup.
- **Future work:** live data connectors (news/port feeds) for early warning, cross-encoder re-ranking, indexing the situation part of narratives only, causal uplift models for strategy selection, authentication and multi-user feedback weighting.